AICreator

Платформа AI-генерации кода для логистической платформы

Апрель 2026 · Проект решения

Разработчики тратят до 70% времени на бойлерплейт — мы это автоматизируем

> *Платформа, которая генерирует код из формальных спецификаций — детерминированно где возможно, с помощью AI только для сложных задач*

- **Сегодня** — каждая команда вручную пишет data-модели, REST-контроллеры, клиенты, скелеты сервисов
- **Возможность** — корпоративная AI-инфраструктура уже развёрнута, но не используется для генерации
- **Наш подход** — 75% детерминированная генерация + 25% AI, только скелет, не бизнес-логика

3 из 4 функций генерации полностью предсказуемы

#	ФУНКЦИЯ	ВХОД	ВЫХОД	ГЕНЕРАТОР
1	Дата-модели	.proto	Классы / структуры	buf (детерм.)
2	REST-эндпоинты	OpenAPI	Контроллеры + middleware	openapi-generator (детерм.)
3	Сервисы из PUML	.puml	Интерфейсы + скелеты	AI — Qwen 3 235B
4	API-клиенты	OpenAPI / AsyncAPI	HTTP / async-клиенты	openapi-gen (детерм.)

Принцип

Почему не «всё на AI»? Потому что enterprise — это предсказуемость

Детерминированная генерация (75%)

- Один вход → всегда один выход
- Работает без GPU и сети
- Мгновенный результат
- Нулевой риск галлюцинаций

AI-генерация (25%)

- Только Функция 3: PUML → сервисы
- Скелет, не бизнес-логика
- Ограниченный контекст промпта
- Fallback на альтернативную модель

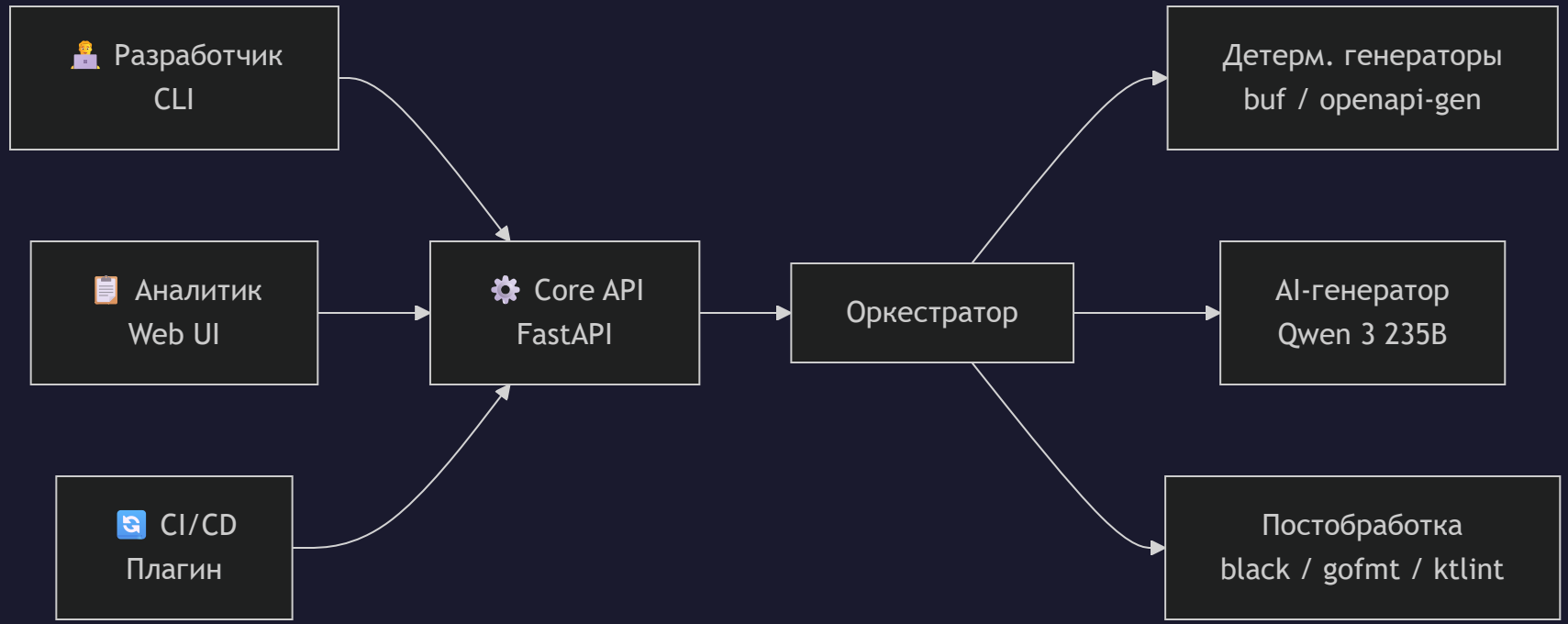
75%

01

Архитектура

Три интерфейса, одно ядро, модульная система

Три интерфейса — один для каждого пользователя



- **CLI** — для разработчиков: `aicreator generate --from proto --to kotlin --spec order.proto`

Модульная архитектура — каждый компонент заменяем

Проверенный open-source стек без vendor lock-in

КОМПОНЕНТ	ТЕХНОЛОГИЯ	GITHUB STARS	РОЛЬ
CLI	typer	16k+	Python CLI-фреймворк
API	FastAPI	80k+	Async REST API
Proto-генерация	buf	13k+	Protobuf тулчейн
OpenAPI-генерация	openapi-generator	23k+	50+ языков
AI-интеграция	litellm	16k+	Model-agnostic прокси
Шаблоны	Jinja2	10k+	Шаблонизатор

Критерий отбора

Мы переиспользуем уже развёрнутую AI-инфраструктуру — новые GPU не нужны

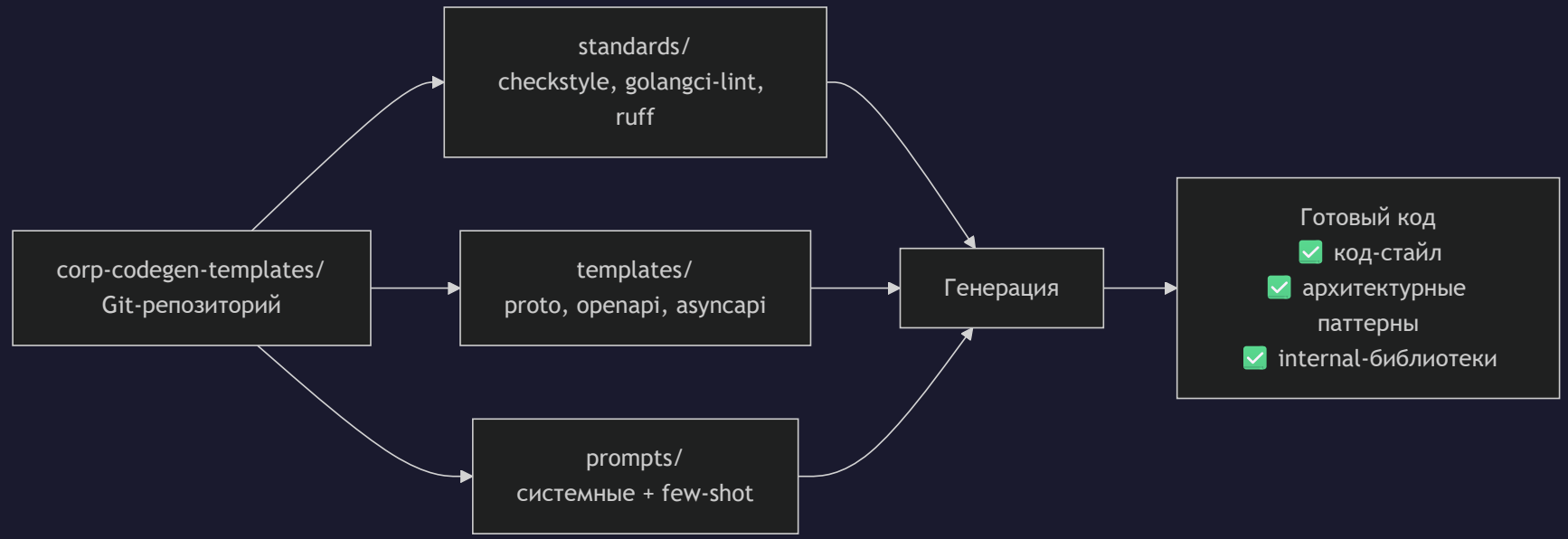
- **Qwen 3 235B** — основная модель генерации
- **GLM 4.7** — fallback-модель для тестов
- OpenAI-compatible API — нативная интеграция
- Смена модели через конфигурацию, без кода

0
НОВЫХ GPU
переиспользуем

2
МОДЕЛИ
Qwen + GLM

⚠ **Model-agnostic**
litellm абстрагирует API — при появлении новых моделей достаточно обновить конфигурацию

Корпоративные стандарты встроены в генерацию — код соответствует с первого раза



- Шаблоны и стандарты версионируются через git tags
- Обновление через PR — полный audit trail

02

Сценарии использования

CLI, Web UI, CI/CD — три пути к одному результату

Разработчик: одна команда → файлы в проекте

1. **Спецификация**
   order.proto в Git
2. **CLI-команда**
   aicreator generate --from proto --to kotlin
3. **Core API**
   Роутинг → buf (детерм.)
4. **Постобработка**
   ktlint форматирование
5. **Результат**
   Файлы в ./generated/

Аналитик через Web UI: загрузка PUML → выбор языка → AI-генерация скелета → скачивание или Merge Request

CI/CD автоматически: обновил OpenAPI в Git → pipeline пересоздал клиенты → автоматический MR

03

Безопасность и качество

Enterprise-grade гарантии на каждом уровне

Безопасность: изолированная среда, полный аудит, без утечек

- **Сеть** — все зависимости во внутренних реестрах, нет внешнего доступа
- **Аутентификация** — SSO (Web UI), API-ключи (CLI/CI)
- **Секреты** — Vault, ротация, без хардкода
- **Аудит** — кто, что, когда, какая модель
- **SAST** — сгенерированный код проходит существующие гейты
- **AI-промпты** — не содержат ПД и коммерческую тайну
- **Rate limiting** — защита от CI/CD-циклов
- **CORS** — только домен Web UI

5 уровней тестирования гарантируют качество генерации

1. **Unit**
   pytest, 80%+ покрытие ядра
2. **Интеграция**
   спека → генерация → компиляция
3. **Golden-файлы**
   регрессия при смене шаблонов
4. **AI-метрики**
   компилируемость, линтинг, оценка
5. **E2E**
   полный цикл CLI/API/UI

> ℹ **Обработка ошибок**
> 4 уровня ошибок: валидация → генерация → AI → постобработка. При отказе AI — функции 1, 2, 4 продолжают работать.

04

Реализация

6 фаз, первый результат через 6 недель

План реализации: от инфраструктуры до пилота за 5–7 месяцев

Инфраструктура
2-3 нед.

Фундамент (CLI + Ф1,Ф4)
4-6 нед.

Полная детерм. генерация
4-6 нед.

AI-генерация (PUML)
4-6 нед.

Web UI + SSO
4-6 нед.

Пилот
4 нед.

6–9
НЕДЕЛЬ
до первого MVP

22–31
НЕДЕЛЬ
полный цикл

1–2
КОМАНДЫ
в пилоте

Ключевые deliverables по фазам

ФАЗА	РЕЗУЛЬТАТ	ПОЛЬЗОВАТЕЛЬ ПОЛУЧАЕТ
0 — Инфраструктура	Реестры, CI/CD, DB	Готовое окружение
1 — Фундамент	CLI + API + proto→модели + клиенты	Генерация через CLI на 1 языке
2 — Детерм.	Все языки + REST + AsyncAPI + CI/CD	Полная детерминированная генерация
3 — AI	PUML→сервисы через Qwen 3	Скелеты сервисов из диаграмм
4 — Web UI	React UI + SSO + мониторинг	Аналитики работают в браузере
5 — Пилот	Метрики, фидбэк, план масштабирования	Подтверждённая ценность

Пилот: 1–2 команды, объективные метрики, решение о масштабировании

Формат пилота

- 1 команда Java/Kotlin + 1 команда Go
- У обеих уже есть proto/OpenAPI спеки
- 4 недели активного использования
- Еженедельный сбор фидбэка

Метрики успеха

ч/нед
СЭКОНОМЛЕНО
на команду

%
БЕЗ ПРАВОК
прошло code review

%
АДОПЦИЯ
регулярное использование

NPS
УДОВЛЕТВОРЁННОСТЬ
пользователей

Ресурсы: компактная команда, существующая инфраструктура

Команда

- **2-3** Python-бэкенд разработчика
- **1** React-фронтенд (с Фазы 4)
- Итого: **3-4 человека**

AI-инфраструктура

- Qwen 3 235B / GLM 4.7 — уже развёрнуты
- **Новые GPU не нужны**

Инфраструктура (as Service)

- PostgreSQL — уже есть
- Redis — уже есть
- Container registry — уже есть
- CI/CD pipeline — уже есть

Для уточнения

- Состав команды разработки
- Выбор пилотных команд
- Детали K8s / Vault / CI/CD

Управляемые риски с конкретными планами митигации

РИСК	ВЕРОЯТНОСТЬ	МИТИГАЦИЯ
AI-генерация ниже ожиданий	Средняя	A/B тестирование моделей, fallback, только скелет
Команды не примут инструмент	Средняя	Гибрид CLI+UI, пилот с мотивированными командами
OSS недоступны в изол. сети	Низкая	Предзагрузка в Фазе 0, проверка до старта
Переполнение контекста LLM	Низкая	Разбиение диаграмм, мониторинг размера

Устойчивость

При полном отказе AI — 3 из 4 функций продолжают работать (детерминированные генераторы)

Что получит компания: скорость, стандарты, масштаб

Ускорение разработки

Автоматизация 60-70% бойлерплейта на логистической платформе

Стандартизация

Весь генерируемый код соответствует корпоративным требованиям с первого раза

Масштабируемость

После пилота — тиражирование на все стримы разработки

Использование AI-инфраструктуры

Практическое применение уже развёрнутых Qwen 3 / GLM 4.7

Решение спроектировано, план готов — нужно одобрение на запуск

Запрос: согласование состава команды и запуск Фазы 0

Следующие шаги:

Согласовать состав команды разработки

Выбрать пилотные команды (Java/Kotlin + Go)

Уточнить инфраструктурные детали (K8s, Vault, CI/CD)

Запуск Фазы 0 — подготовка инфраструктуры